# Laboratorio No.1 – Semana 16 - Swagger y Health Check para WebStorageSample

**Nombre**: Camila Raquel Marcos Medinilla **Carné**: 1120525

**Objetivo:** Configurar Swagger para documentar endpoints HTTP.

- Crear GET /health para comprobar que la aplicación responde.
- Crear GET /api/info para publicar información general de la aplicación.
- Probar los endpoints desde Swagger y verificar sus respuestas HTTP/JSON.
- Mantener funcionando la página Razor de carga de imágenes y registrar los cambios en Git.

**Repositorio**
Utilice como punto de partida el repositorio de actividad 1 semana 14
**Basado en GitHub:** https://github.com/Azure-Samples/serviceconnector-webapp-storageblogb-dotnet

## Actividad

**Paso 1: revisar el paquete y Startup.cs**

Desde la carpeta donde está el archivo .csproj, verifica el paquete:

```
dotnet list package
```

Si Swashbuckle.AspNetCore no aparece instalado, agrega el paquete con:

```
dotnet add package Swashbuckle.AspNetCore
```

En Startup.cs, el método ConfigureServices debe incluir los servicios siguientes. Conserva cualquier otro servicio que ya necesite el proyecto:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddRazorPages();
    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen();
}
```

No dupliques AddRazorPages() ni AddSwaggerGen() si ya existen. AddEndpointsApiExplorer() ayuda a descubrir endpoints definidos con MapGet

---

**Paso 2: habilitar Swagger**

Dentro de Configure(), habilita Swagger en desarrollo. Coloca este bloque después del manejo de errores y antes de UseHttpsRedirection():

```
if (env.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

La interfaz se abrirá manualmente en /swagger. No es necesario configurar el navegador para que Swagger se abra automáticamente.

---

**Paso 3: crear los endpoints**

Localiza el bloque app.UseEndpoints(...) al final de Configure(). Conserva endpoints.MapRazorPages() y agrega los dos endpoints dentro del mismo bloque:

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapRazorPages();

    endpoints.MapGet("/health", () =>
        Results.Ok(new
        {
            status = "Healthy",
            timestamp = DateTime.UtcNow
        }))
        .WithName("GetHealth")
        .WithTags("Health")
        .WithSummary("Verifica que la aplicación responde");

    endpoints.MapGet("/api/info", () =>
        Results.Ok(new
        {
            application = "WebStorageSample",
            description = "Aplicación de ejemplo para Azure Blob Storage",
            version = "1.0.0"
        }))
        .WithName("GetApplicationInfo")
        .WithTags("Application")
        .WithSummary("Obtiene información general de la aplicación");
});
```

Si el compilador no reconoce Results, agrega using Microsoft.AspNetCore.Http; al inicio de Startup.cs.
No elimines el resto de las instrucciones de Configure().

---

**Paso 4: compilar y ejecutar**

```
dotnet clean
dotnet restore
dotnet build
dotnet run
```

Si la compilación falla, corrige los errores antes de continuar. Usa el puerto HTTPS que indique la terminal; en el ejemplo de clase se utilizó https://localhost:5001.

---

**Paso 5: probar desde Swagger**

Abre en el navegador: https://localhost:5001/swagger
Debes encontrar los grupos Health y Application. Ejecuta cada operación con Try it out y Execute.

### 5.1 GET /health

Respuesta esperada: HTTP 200 OK y un JSON similar a este:

```
{
  "status": "Healthy",
  "timestamp": "2026-10-..."
}
```

### 5.2 GET /api/info

Respuesta esperada: HTTP 200 OK y un JSON similar a este:

```
{
  "application": "WebStorageSample",
  "description": "Aplicación de ejemplo para Azure Blob Storage",
  "version": "1.0.0"
}
```

---

## Preguntas de análisis

1. **¿Por qué Swagger mostraba "No operations defined in spec!" antes de crear los endpoints?**
   Porque la app todavía no tenía ninguna ruta creada. Swagger es como un menú de restaurante: si la cocina no ha definido ningún platillo, el menú sale vacío. Al crear los endpoints, aparecieron.
2. **¿Qué diferencia existe entre /health y /api/info?**
   /health responde "estoy vivo y funcionando", como tomarle el pulso a la app. /api/info da datos de la app, como su nombre y versión, igual que una ficha de presentación.
3. **¿Qué comprueba el endpoint /health y qué no comprueba?**
   Comprueba que la app está encendida y puede responder. No comprueba que pueda conectarse a otros servicios, como Blob Storage. Es como confirmar que una tienda abrió sus puertas, sin saber si tiene mercancía en la bodega.
4. **¿Qué función cumplen AddSwaggerGen() y AddEndpointsApiExplorer()?**
   AddEndpointsApiExplorer() es el que "descubre" qué rutas existen, como alguien que hace el inventario de los platillos. AddSwaggerGen() toma ese inventario y arma la página bonita de Swagger para verlo y probarlo.
5. **¿Qué sería necesario implementar para comprobar también el acceso real a Azure Blob Storage?**
   Hacer que /health intente de verdad hablar con Blob Storage, por ejemplo pedirle la lista de archivos del contenedor o preguntar si existe. Si responde, todo bien. Si falla, /health avisa que hay un problema. Sería como no solo ver que la tienda abrió, sino mandar a alguien a confirmar que la bodega tiene llave y mercancía.

**Entrega**

Dentro del repositorio de GitHub, cree la carpeta semana-16/lab1/ y coloque en ella un PDF con la solución del laboratorio.
Enviar link de github a plataforma de la universidad.